

CODE & KNOWLEDGE INTELLIGENCE

Wissen wird navigierbar.

Doctus macht gewachsene COBOL-Anwendungen, begleitende Dokumente und ihre Zusammenhänge für Teams verständlich, durchsuchbar und nutzbar.

PRODUKT

On-Premise Knowledge Intelligence

ZIELBILD

Legacy-Systeme sicher
verstehen und weiterentwickeln

confidential

Ein Arbeitsraum - alles im Blick.

Doctus bündelt Programmcode, technische Dokumentation und Projektwissen an einem Ort. Statt Informationen in Dateien, Tickets, Wikis und Köpfen zu suchen, können Teams Fragen stellen, Quellen prüfen und Zusammenhänge direkt verfolgen.

FOKUS COBOL- und Legacy-Bestände verstehen	BEREITSTELLUNG Im eigenen Rechenzentrum oder Server
QUELLEN Code, Dokumente, Wikis, Tickets und Ablagen	ARBEITSWEISE Fragen, finden, nachvollziehen und teilen

01 / ANBINDEN Wissen verbinden Projekte erhalten ihre Code- und Wissensquellen – von Git bis zur Dokumentenablage.	02 / ERSCHLIESSEN Strukturen erkennen Doctus verarbeitet Dateien, Codeelemente, Referenzen und fachliche Dokumente zu einer belastbaren Wissensbasis.	03 / ERKUNDEN Zusammenhänge sehen Suche, Graphen, Referenzen und Codeansichten machen Abhängigkeiten greifbar.	04 / HANDELN Fundiert entscheiden Antworten bleiben überprüfbar, weil ihre Herkunft bis in die konkrete Quelle sichtbar bleibt.
--	---	--	---

Aus verteiltem Wissen wird eine gemeinsame Arbeitsgrundlage.

01

Schneller einarbeiten

Neue Kolleginnen und Kollegen finden Programme, Begriffe, Datenfelder und fachliche Hintergründe nachvollziehbar vor – ohne auf einzelne Wissensträger angewiesen zu sein.

02

Änderungen sicherer planen

Wer die Auswirkungen einer Anpassung verstehen muss, verfolgt Aufrufe, Abhängigkeiten und zugehörige Dokumentation über mehrere Perspektiven hinweg.

03

Wissen bewahren

Code und Begleitwissen bleiben im Zusammenhang auffindbar, auch wenn Teams, Zuständigkeiten oder Systemlandschaften sich verändern.

Die zentrale Idee: Doctus ersetzt keine fachliche Prüfung. Es verkürzt aber den Weg von einer Frage zu nachvollziehbaren, relevanten Quellen erheblich.

Der Zugang beginnt kontrolliert und bleibt nachvollziehbar.

Doctus ist für den Betrieb mit eigenen Benutzerkonten ausgelegt. Nach der Anmeldung sehen Personen nur die Teams und Projekte, für die sie freigeschaltet sind.

01 Anmelden und Passwort ändern Beim ersten Zugang kann ein Passwortwechsel erzwungen werden. Fehlversuche werden begrenzt und Konten bei auffälligen Anmeldeversuchen zeitweise geschützt.	02 Benutzer verwalten Administratoren legen Konten an, aktivieren oder deaktivieren sie, setzen Kennwörter zurück und heben Sperren gezielt auf.
03 Teams zuordnen Teams bilden organisatorische Zuständigkeiten ab. Eine Person kann mehreren Teams angehören; die Administration behält den Gesamtüberblick.	04 Projektzugriff steuern Innerhalb eines Teams kann der Zugang zusätzlich pro Projekt vergeben oder beantragt werden. So bleibt auch bei gemeinsamen Teams die Sicht auf konkrete Bestände kontrollierbar.

Klare Zuständigkeiten für Wissen, das zusammengehört.



Teams

Teams bündeln Personen nach Organisation, Produktlinie oder Aufgabe. Sie sind der erste Schutzraum für Inhalte und erleichtern es, gemeinsame Arbeitsbereiche sinnvoll zu strukturieren.

Administratoren verwalten Teams und Mitgliedschaften zentral. Personen ohne Teamzuordnung erhalten keinen unbeabsichtigten Einblick in Bestände.



Projekte

Ein Projekt ist der fachliche und technische Rahmen für zusammengehörige Quellen: etwa eine Anwendung, einen Modernisierungsstrang oder einen abgegrenzten Systembestand.

Projekte besitzen Mitglieder und Rollen. Berechtigte Personen können weitere Mitglieder aufnehmen; andere Teammitglieder können einen Zugriff anfragen, sofern das Projekt für sie sichtbar ist.

Doctus bringt Code und Begleitwissen in einen gemeinsamen Kontext.

Wissensquellen werden projektspezifisch eingerichtet, synchronisiert und im Hintergrund verarbeitet. Der Fortschritt und mögliche Hinweise bleiben im Job-Center und an der jeweiligen Quelle sichtbar.

CODE Git-Repositories GitHub, GitLab, Bitbucket und generische Git-Server lassen sich inklusive Branch auswählen. Auch große, mehrteilige Bestände können getrennt und wiederaufsetzbar synchronisiert werden.	DOCS Dokumente und Ablagen Datei-Uploads, WebDAV und Ordner- bzw. NAS-Quellen erschließen Dokumente wie PDF, Word, Markdown und Textdateien. Änderungen werden beim Folgelauf gezielt erkannt.	WORK Wikis und Tickets Confluence und Jira ergänzen Code um Entscheidungswissen, Anforderungen, Tickets und fachliche Erläuterungen – dort, wo dieses Wissen in der Praxis entsteht.
--	--	--

Für geteilte Ablagen: Ordnerquellen werden ausschließlich lesend eingebunden. Doctus erkennt neue, geänderte und gelöschte Dateien und aktualisiert nur den betroffenen Teil der Wissensbasis.

Eine Frage kann sich in mehrere, gleichzeitig sichtbare Perspektiven entfalten.

Der Arbeitsraum lässt sich als ein bis vier Bereiche anordnen. Teams kombinieren etwa Chat, Quellcode, Referenzen und Graphen – ohne den fachlichen Kontext beim Wechsel zwischen Ansichten zu verlieren.

01 Seitenleiste als Orientierung

Projekte, Hauptansichten, Suche, Job-Center und Einstellungen bleiben direkt erreichbar.

02 Fokusobjekt als roter Faden

Ein Programm, eine Sektion oder ein Dokument kann in den Mittelpunkt gestellt werden. Die umliegenden Ansichten beziehen sich darauf.

03 Zeilenreferenzen als Beleg

Chat- und Rechercheergebnisse führen über anklickbare Quellenangaben an die passende Stelle im Original zurück.

04 Layout nach Aufgabe

Die Anordnung der Bereiche kann an Analyse, Recherche oder konzentriertes Lesen angepasst werden.

Vom Quelltext zur verständlichen Struktur.

01

COBOL verstehen

Doctus erkennt unter anderem Programme, Copybooks, Sektionen, Paragraphen, Datenfelder, Dateibeschreibungen und eingebettete SQL-Bereiche. Physische Zeilen bleiben dabei erhalten.

02

Beziehungen verfolgen

Aufrufe, Sprünge, Ausführungsfolgen, Copybook-Verwendungen sowie Lese- und Schreibzugriffe werden als nachvollziehbare Beziehungen erschlossen. Dynamische oder nicht auflösbare Bezüge werden sichtbar markiert.

03

Fragen mit Quellen stellen

Der Chat nutzt ausschließlich die für das Projekt zugängliche Wissensbasis als Kontext. Antworten können zu Codezeilen und Dokumentstellen zurückführen, damit Aussagen geprüft werden können.

KI verkürzt den Weg zur Antwort – die Quelle bleibt maßgeblich.

Doctus verbindet lokale Sprachmodelle mit dem jeweils berechtigten Projektwissen. Antworten werden an vorhandenen Code und Dokumente gebunden; sie ersetzen weder fachliche Prüfung noch eine Freigabeentscheidung.

01 Fragen beantworten Teams stellen Fragen in natürlicher Sprache zu Programmen, Datenfeldern, Dokumenten und Fachbegriffen. Doctus sucht die relevanten Stellen und beantwortet die Frage im Projektkontext.	02 Code erklären Ein fokussiertes Programm, eine Sektion oder eine Zeile lässt sich erläutern. Das ist besonders wertvoll beim Onboarding und wenn selten gepflegte Abläufe wieder aufgenommen werden.	03 Auswirkungen einordnen Chat, Strukturinformationen und Graphen helfen, von einer Änderung zu betroffenen Aufrufen, Datenzugriffen und zugehöriger Dokumentation zu navigieren.
04 Wissen verdichten Verteilte technische und fachliche Quellen werden für Recherche zugänglich. Quellenkontext und hinterlegte Fachbegriffe helfen dem Modell, kundenspezifischen Jargon richtig einzuordnen.	05 Antworten belegen Antworten enthalten Referenzen auf die tatsächlich herangezogenen Dateien, Dokumentstellen oder Codezeilen. Damit kann jede Aussage am Original geprüft werden.	06 Regeln prüfen Der Code-Compliance-Checker kann einen definierten Prüfumfang mit einem lokalen Sprachmodell bewerten. Das Ergebnis ist ein Hinweis für die Prüfung, kein automatischer Befund.

Modelle nach Aufgabe und Hardware auswählen.

AUFGABE	EMPFEHLUNG	EINSATZENTSCHEIDUNG
Retrieval und Index	BGE-M3 als Embedding-Modell	Doctus-Standard für mehrsprachige, semantische Suche. Auch ohne generatives Modell bleiben Import, Index und Suche nutzbar.
Chat und Compliance	Mistral NeMo 12B Instruct, Q4_K_M	Im Doctus-Stack validierte Standardoption für GPU-Hosts ab 16 GB RAM und 8 GB VRAM. Für CPU-only-Piloten nicht empfohlen.
Höhere Prüfgenaugkeit	Mistral NeMo 12B Instruct, Q8_0	Bei mindestens 24 GB VRAM für den Compliance-Checker; benötigt mehr Speicher, reduziert im validierten Doctus-Test aber übersehene Befunde.
Mehrstufige Analyse	Magistral Small als Reasoning-Modell	Geeignet für komplexe Prüf- und Planungsfragen. Die Ollama-Anbindung wurde technisch erprobt: Mit deaktiviertem Reasoning-Modus bleiben strukturierte JSON-Ergebnisse zügig verarbeitbar. Vor Produktiveinsatz sind Qualität, Laufzeit und Speicherbedarf am echten Bestand zu bewerten.

Betriebsprinzip: Das Embedding-Modell und das Chat-Modell laufen über Ollama im eigenen Netzwerk. Ein Cloud-Modell ist optional, aber nur nach expliziter administrativer Freigabe vorgesehen.

Was als Nächstes sinnvoll ergänzt werden kann.

A

Änderungsvorhaben vorbereiten

Aus einem Fachwunsch betroffene Programme, Regeln und offene Fragen zusammenstellen – mit Freigabe durch Fachbereich und Entwicklung.

B

Regeln und Begriffe extrahieren

Wiederkehrende Geschäftsregeln, Datenbegriffe und Abkürzungen als überprüfbare Vorschläge für eine kuratierte Wissensbasis ableiten.

C

Semantische Änderungen prüfen

Änderungen zwischen Ständen nicht nur zeilenweise, sondern nach betroffenen Abläufen, Schnittstellen und Dokumentation priorisieren.

Spezifische Ansichten für jede Situation.

Ansichten sind die Bereiche, die im Arbeitsraum geöffnet und nebeneinander angeordnet werden können. Bis zu vier Ansichten lassen sich gleichzeitig nutzen; Chat und Code-Editor sind dabei eigenständige Ansichten.

ANSICHT 01

Chat

ANSICHT 02

Code-Editor

ANSICHT 03

Dokumentation

ANSICHT 04

Wissensnetz

ANSICHT 05

Call-Graph

ANSICHT 06

Web-Vorschau

ANSICHT 07

Link Manager

Funktionalität innerhalb der Ansichten

ANSICHT 01 • CHAT

Fragen stellen und Antworten belegen

Der Chat ist die dialogorientierte Rechercheoberfläche für den jeweils gewählten Projektkontext. Er nimmt Fragen in natürlicher Sprache entgegen und nutzt ausschließlich die Wissensquellen, auf die die angemeldete Person im Projekt zugreifen darf.

Antworten enthalten Quellenangaben. Mit einem Klick lässt sich von einer Aussage direkt zur betreffenden Codezeile oder Dokumentstelle wechseln; ein Programm, ein Dokument oder eine markierte Codezeile kann außerdem gezielt als Kontext für die nächste Frage festgelegt werden.

Fragen im Projektkontext

Quellenchips mit Zeilensprung

Code oder Dokument als Fokus anheften

Chat-Verlauf teilen und fortführen

Code lesen, navigieren und prüfen

Der Code-Editor stellt Programmdateien als prüfbare Originalquelle dar. Dateien, erkannte Codeelemente und konkrete Zeilen lassen sich ansteuern, sodass die technische Grundlage einer Recherche ohne Medienbruch erreichbar bleibt.

Aus dem Editor heraus können Referenzen und zugehörige Strukturelemente verfolgt werden. Eine ausgewählte Zeile oder ein Codeobjekt kann an den Chat übergeben werden, um Fragen genau auf den relevanten Ausschnitt einzugrenzen.

Code und Strukturelemente lesen

Zu konkreten Zeilen navigieren

Referenzen im Kontext prüfen

Codeausschnitte für den Chat fokussieren

Begleitwissen neben dem Code nutzen

Die Dokumentationsansicht öffnet erschlossene Dokumente direkt im Arbeitsraum. Fachliche Beschreibungen, technische Unterlagen und weitere Inhalte aus angebundenen Wissensquellen bleiben damit bei der Codeanalyse verfügbar.

Dokumentstellen können als Herkunft einer Chat-Antwort geöffnet und im ursprünglichen Zusammenhang gelesen werden. Der Wechsel zwischen Dokumentation und Code unterstützt dabei, fachliche Aussagen an ihrer technischen Umsetzung zu prüfen.

Dokumente im Kontext lesen

Quellenstellen direkt öffnen

Zwischen Dokument und Code wechseln

Dokumente als Beleg nutzen

Zusammenhänge über Quellgrenzen hinweg erkunden

Das Wissensnetz visualisiert Verbindungen zwischen Codeelementen, Dokumenten und kuratierten Beziehungen. Es hilft, die Reichweite eines Begriffs, einer fachlichen Regel oder eines technischen Bausteins über mehrere Quellen hinweg zu erfassen.

Knoten lassen sich auswählen und als neuer Fokus in den Arbeitsraum übernehmen. Dadurch kann eine Untersuchung vom Überblick schrittweise zu einem konkreten Objekt, seiner Dokumentation oder seinem Code verzweigen.

Code- und Wissensknoten gemeinsam

Verbindungen kontextuell erkunden

Auswahl als neuer Fokus

Reichweite von Themen nachvollziehen

Technische Ablaufketten verfolgen

Der Call-Graph zeigt Aufrufe zwischen Programmen und weiteren Codeelementen. So werden technische Ablaufketten, Abhängigkeiten und mögliche Auswirkungen einer Änderung sichtbar, ohne die Beziehungen einzeln aus Dateien zusammensuchen zu müssen.

Die gewünschte Tiefe kann eingestellt und die Anzeige nach Beziehungstyp eingegrenzt werden. Dynamische oder nicht vollständig auflösbare Verbindungen werden erkennbar gehalten; für die Weiterverarbeitung lässt sich der Graph als JSON, CSV oder GraphML exportieren.

Ein bis drei Beziehungsebenen

Filter nach Beziehungstyp

Dynamische Bezüge sichtbar halten

Export als JSON, CSV oder GraphML

Webquellen im Ursprungskontext lesen

Die Web-Vorschau zeigt Inhalte aus angebundenen Webquellen in einer eigenen Ansicht. Sie bewahrt beim Recherchieren den Bezug zur ursprünglichen Webseite, statt den Inhalt nur als losgelösten Suchtreffer zu behandeln.

Die Ansicht kann parallel zu Chat, Code oder Dokumentation geöffnet werden. Damit lässt sich der Ursprung eines Webinhalts mit den daraus erschlossenen Informationen und den übrigen Projektquellen abgleichen.

Webquellen im Arbeitsraum öffnen

Ursprung und Inhalt zusammen betrachten

Neben weiteren Ansichten verwenden

In den Kontext zurückwechseln

Wichtige Beziehungen kuratieren

Der Link Manager dient dazu, fachlich bedeutsame Beziehungen zwischen Code und Dokumentation sichtbar zu machen, zu prüfen und zu pflegen. Er ergänzt automatisch erkannte Zusammenhänge dort, wo menschliche Einordnung und dauerhafte Verbindlichkeit wichtig sind.

Zusätzlich können Themen relevante Knoten unabhängig von einem einzelnen Repository bündeln. So entsteht eine geteilte Orientierung für wiederkehrende Fragestellungen, Fachbegriffe oder Modernisierungsvorhaben.

Verbindungen prüfen und pflegen

Code und Dokumentation gezielt verknüpfen

Themenbereiche strukturieren

Geteilte Orientierung schaffen

Nicht alles, was Doctus leistet, ist eine eigene Ansicht.

F01

Globale Suche

Die globale Suche führt über Projekte hinweg zu passenden Codeelementen und Wissensdokumenten – stets innerhalb der persönlichen Berechtigung. Ergebnisse öffnen sich anschließend im passenden Arbeitskontext.

F02

Abläufe nachvollziehen

Aufrufe, Sprünge, Ausführungsfolgen und Datenzugriffe helfen dabei, technische Prozesse zu verstehen. Je nach Fragestellung werden sie im Code geprüft oder im Call-Graph als Ablaufkette verfolgt.

F03

Verarbeitung beobachten

Import, Synchronisation und Erschließung laufen im Hintergrund. Das Job-Center informiert über Fortschritt, Zustand und Hinweise zu diesen Vorgängen.

Konfiguration dort, wo sie gebraucht wird.

BEREICH	WOFÜR ER DIENT
Projekte	Projekte anlegen, beschreiben, Mitglieder und Zugangsanfragen verwalten.
Teams und Benutzer	Organisationseinheiten, Mitgliedschaften und Benutzerkonten administrieren.
Wissensquellen	Verbindungen einrichten, Synchronisationen anstoßen und ihren Zustand nachvollziehen.
KI-Einstellungen	Die lokal verfügbare KI konfigurieren; optionale externe Modelle bleiben eine bewusste Freigabeentscheidung.
Layout	Den Arbeitsraum passend zur Aufgabe einrichten – von fokussiert bis mehrspaltig.
Protokolle und Diagnose	Für Betrieb und Fehlersuche stehen bereinigte Diagnosen und begrenzte Protokolle zur Verfügung.

Für den kontrollierten Betrieb in der eigenen Umgebung.

Doctus ist standardmäßig als On-Premise-Lösung ausgelegt. Der Datenbestand, die Quellen und die KI-Verarbeitung bleiben im eigenen Betriebsumfeld. Eine Cloud-Anbindung für Sprachmodelle ist nur nach expliziter Freigabe vorgesehen.

ZUGANG	ANWENDUNG	VERARBEITUNG	WISSEN	KI
Web-Arbeitsplatz Browserbasierte Oberfläche für Recherche, Analyse und Administration.	Fachlogik Steuert Zugriffe, Projekte, Suche, Chat und die Bedienoberfläche.	Hintergrunddienste Synchronisieren Quellen, analysieren Dateien und bauen Beziehungen auf.	Daten- und Vektorspeicher Hält Strukturinformationen, Dokumentinhalte, Referenzen und Suchrepräsentationen vor.	Lokale Inferenz Erstellt Suchrepräsentationen und beantwortet Fragen mit lokal betriebenen Modellen.

AUTH Zugangsschutz Lokale Konten, sichere Kennwortspeicherung, erzwungene Passwortwechsel, Sperrmechanismen und Team-/Projektberechtigungen schützen den Zugriff.	DATA Datensouveränität Quellen, Index und KI laufen standardmäßig im eigenen Netzwerk. Geheimnisse von angebundenen Quellen werden verschlüsselt abgelegt.	OPS Betriebssicherheit Gesundheitsprüfungen, begrenzte Logdateien, bereinigte Diagnoseinformationen und voneinander getrennte Dienste unterstützen einen wartbaren Betrieb.
--	---	--

Von der ersten Quelle bis zur fundierten Antwort.

01 Organisation und Projekt einrichten

Ein Administrator richtet Teams, Benutzer und ein Projekt für den zu erschließenden Bestand ein.

02 Wissensquellen verbinden

Die verantwortliche Person bindet Code-Repositories, Dokumente, Wissensplattformen oder Ablagen an und stößt die erste Verarbeitung an.

03 Bestand erkunden

Nach der Verarbeitung kann das Team über Suche, Codeansicht und Graphen orientieren: Was gibt es? Was hängt zusammen? Welche Dokumentation gehört dazu?

04 Fragen nachvollziehbar beantworten

Im Chat entstehen Antworten im Kontext des ausgewählten Projekts. Quellenreferenzen ermöglichen den direkten Sprung zur überprüfbaren Grundlage.

05 Wissen gemeinsam nutzbar halten

Neue Quellen, aktualisierte Dokumente und Änderungen am Bestand werden erneut synchronisiert. Wichtige Verbindungen lassen sich kuratieren und als Themen organisieren.

Legacy-Systeme werden nicht einfacher – aber wesentlich besser zugänglich.

Doctus gibt Teams eine gemeinsame Sprache für komplexe Bestände. Es verbindet die Genauigkeit des Originalmaterials mit einer schnellen, modernen Recherche- und Navigationsoberfläche.

Dadurch lässt sich fachliches und technisches Wissen früher in Entscheidungen einbeziehen: beim Onboarding, bei der Fehleranalyse, vor einer Änderung, bei Übergaben und bei der schrittweisen Modernisierung. Die entscheidende Grundlage bleibt dabei sichtbar: der konkrete Code und die zugehörige Dokumentation.

01 Überblick Was Doctus ist	03 Zugang Benutzer und Rechte	05 Wissensquellen Code und Dokumente
06 Arbeitsraum Recherche im Kontext	08 Lokale KI Use Cases und Modelle	09 Ansichten Die passenden Perspektiven
11 Technik Aufbau und Schutz	ANHANG Produktbilder Arbeitsraum im Einsatz	

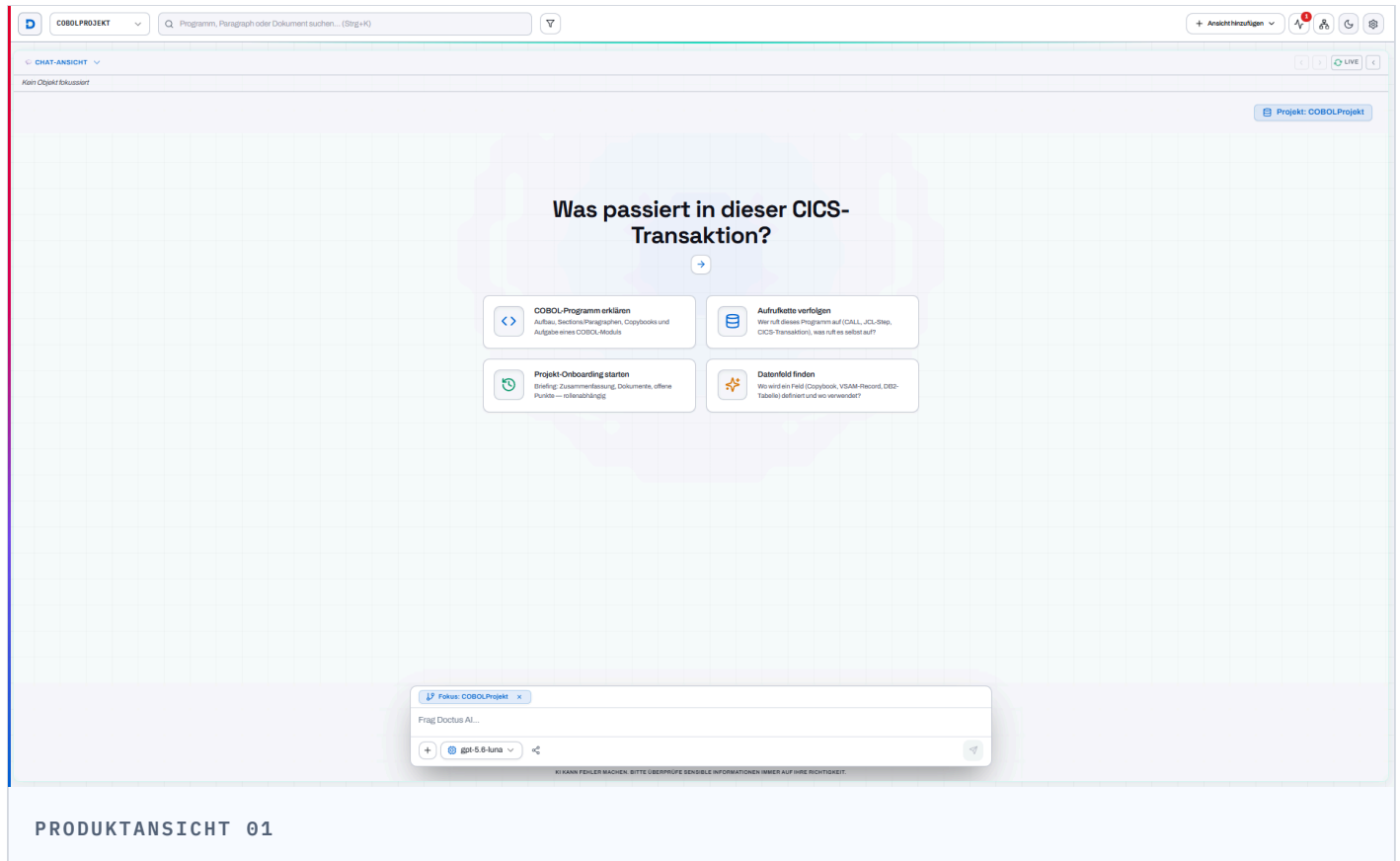
 ANHANG • PRODUKTBILDER

Doctus - Vorschau

Hinweis: Die folgenden Ansichten zeigen eine Vorschau und stellen noch nicht das fertige Produkt dar.

Die Anwendung in der Praxis.

Die folgenden Ansichten zeigen zentrale Arbeitsbereiche von Doctus im Zusammenspiel.



COBOLPROJEKT

id

Ansicht hinzufügen

CHAT-ANSICHT

Kein Objekt ausgewählt

CODE

000-START
S01JAHRRESALZUEG.dbl

000-START
S01PLANSR00.dbl

000-START
S01LEGACHTKONV.dbl

000-START
S01KONTOTABELLE.dbl

000-START
S01STAPELVERARBEITUNG.dbl

000-START
S01ZNR0BERECHNUNG.dbl

+70 weitere — Suche eingetrennt

DOKUMENTE

STAPELVERARBEITUNG.dbl

KONTOSTATUSBEREICHUNG.dbl

Welche JCL-Steps laufen in dieser Kette?

COBOL-Programm erklären
Aufbau, Sections/Paragraphen, Copybooks und
Aufgabe eines COBOL-Moduls

Auftragskette verfolgen
Wer ruft dieses Programm auf (CALL, JCL-Step,
CICS-Transaktion), was ruft es selbst auf?

Projekt Onboarding starten
Briefing, Zusammenfassung, Dokumente, offene
Punkte — rollenspezifisch

Datenfeld finden
Wo wird ein Feld (Copybook, VSAM-Record, DB2-
Tabelle) definiert und wo verwendet?

Fokuse: COBOLProjekt

Frag Doctus AI...

+ gpt-5.5-luna

PRODUKTANSICHT 02

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

CHAT-ANSICHT

Projekt: COBOLProjekt

Welche JCL-Steps laufen in dieser Kette?

COBOL Programm erklären

Aufbau, Sections/Paragraphen, Copybooks und Aufgabe eines COBOL-Moduls

Aufrufkette verfolgen

Wie ruft dieses Programm auf (CALL, JCL-Step, CICS-Transaktion), was ruft es selbst auf?

Projekt Onboarding starten

Briefing, Zusammenfassung, Dokumente, offene Punkte – rollenabhängig

Datenfeld finden

Wie wird ein Feld (Copybook, VSAM-Record, DB2-Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: 000-START

Frag Docus AI...

gpt-5.6-luna

Kleiner Fehler-Banner: KLEINER FEHLER BACHEN. BITTE ÜBERPRÜFE SENSIBLE INFORMATIONEN IMMER AUF IHRE RICHTIGKEIT!

CODE-EDITOR

src/FILEPROG.cbl

```
1 ..... IDENTIFICATION DIVISION.
2 PROGRAM-ID. FILEPROG.
3 AUTHOR. GEMINI-CLT.
4
5 ENVIRONMENT DIVISION.
6 INPUT-OUTPUT SECTION.
7 FILE-CONTROL.
8     SELECT USER-FILE ASSIGN TO "data/users.dat"
9     ORGANIZATION IS LINE SEQUENTIAL.
10
11 DATA DIVISION.
12 FILE SECTION.
13     COPY "FILE-REC.cpy".
14
15 WORKING-STORAGE SECTION.
16 01 MS-EOF-SWITCH      PIC X(01) VALUE 'N'.
17 01 88 END-OF-FILE     VALUE 'Y'.
18
19 LINKAGE SECTION.
20 01 LK-ACTION          PIC X(05).
21 01 LK-STATUS         PIC 9(02).
22
23 PROCEDURE DIVISION USING LK-ACTION LK-STATUS.
24
25 MAIN-LOGIC SECTION.
26 000-START.
27     IF LK-ACTION = "READ"
28         PERFORM 100-READ-FILE
29     ELSE
30         MOVE 99 TO LK-STATUS
31     END-IF.
32     GOBACK.
33
34 READ-SECTION SECTION.
35 100-READ-FILE.
36     OPEN INPUT USER-FILE.
37     MOVE 00 TO LK-STATUS.
38
39     PERFORM UNTIL END-OF-FILE
40         READ USER-FILE
41         AT END
42             MOVE 'Y' TO MS-EOF-SWITCH
43         NOT AT END
44             DISPLAY "FILEPROG: READ " FD-USER-NAME
45         END-READ
46     END-PERFORM.
47
48     CLOSE USER-FILE.
49
```

SEQUENZ (1-4)

PRODUKTANSICHT 03

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

CHAT-ANSICHT

000-START - PARAGRAPH

Projekt: COBOLProjekt

Welche JCL-Steps laufen in dieser Kette?

COBOL Programm erklären
Aufbau, Sections/Paragraphen, Copybooks und Aufgabe eines COBOL-Moduls

Aufrufkette verfolgen
Wer ruft dieses Programm auf (CALL „JCL-Step“, CICS-Transaktion), was ruft es selbst auf?

Projekt Onboarding starten
Briefing, Zusammenfassung, Dokumente, offene Punkte – rollenabhängig

Datenfeld finden
Wo wird ein Feld (Copybook, VSAM-Record, DB2-Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: 000-START

Was macht dieses Programm?

+ gpx-5.6-luna

KANN FEHLER BACHEN. BITTE ÜBERPRÜFE SENSIBLE INFORMATIONEN IMMER AUF IHRE RICHTIGKEIT!

COBOL

src/FILEPROG.cbl

000-START - PARAGRAPH

SEQUENZ (1-4)

```
1 ..... IDENTIFICATION DIVISION.
2 PROGRAM-ID, FILEPROG.
3 AUTHOR, GEMINI-CLT.
4
5 ENVIRONMENT DIVISION.
6 INPUT-OUTPUT SECTION.
7 FILE-CONTROL.
8     SELECT USER-FILE ASSIGN TO "data/users.dat"
9       ORGANIZATION IS LINE SEQUENTIAL.
10
11 DATA DIVISION.
12 FILE SECTION.
13     COPY "FILE-REC.cpy".
14
15 WORKING-STORAGE SECTION.
16 01 MS-EOF-SWITCH      PIC X(01) VALUE 'N'.
17 08 END-OF-FILE        VALUE 'Y'.
18
19 LINKAGE SECTION.
20 01 LK-ACTION           PIC X(05).
21 01 LK-STATUS          PIC 9(02).
22
23 PROCEDURE DIVISION USING LK-ACTION LK-STATUS.
24
25 MAIN-LOGIC SECTION.
26 000-START.
27 IF LK-ACTION = "READ"
28     PERFORM 100-READ-FILE
29 ELSE
30     MOVE 99 TO LK-STATUS
31 END-IF.
32 GOBACK.
33
34 READ-SECTION SECTION.
35 100-READ-FILE.
36 OPEN INPUT USER-FILE.
37 MOVE 00 TO LK-STATUS.
38
39 PERFORM UNTIL END-OF-FILE
40     READ USER-FILE
41     AT END
42         MOVE 'Y' TO MS-EOF-SWITCH
43     NOT AT END
44         DISPLAY "FILEPROG: READ " FD-USER-NAME
45     END-READ
46 END-PERFORM.
47
48 CLOSE USER-FILE.
49
```


COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

Ansicht hinzufügen

NEUER CHAT

VERLAUF

Was steht in der README?

Was macht dieses Program?

Was macht FIN-DATA.cpy?

Was ist das für ein Projekt?

hallo

WISSENSQUELLEN

Keine Wissensquellen angepinnt

Administrator

Lukas Stang

CHAT-ANSICHT

Kein Objekt fokussiert

Projekt: COBOLProjekt

Wo wird auf diese DB2-Tabelle zugegriffen?

COBOL-Programm erklären

Aufbau, Sections/Paragraphen, Copybooks und Aufgabe eines COBOL-Moduls

Auflaufkette verfolgen

Wer ruft dieses Programm auf (CALL „JCL-Step“, CICS-Transaktion), was ruft es selbst auf?

Projekts Onboarding starten

Briefing, Zusammenfassung, Dokumente, offene Punkte — rollenabhängig

Datenfeld finden

Wo wird ein Feld (Copybook, VSAM-Record, DB2-Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: KONTO-DATEN.cpy.d

Frag Doctus AI...

gpt-5.6-luna

WIKI: KEIN FEHLER BACHEL. BITTE ÜBERPRÜFE SENSIBLE INFORMATIONEN WERD AUF IHRE RICHTIGKEIT.

WISSENSNETZ (GRAPH)

Kein Objekt fokussiert

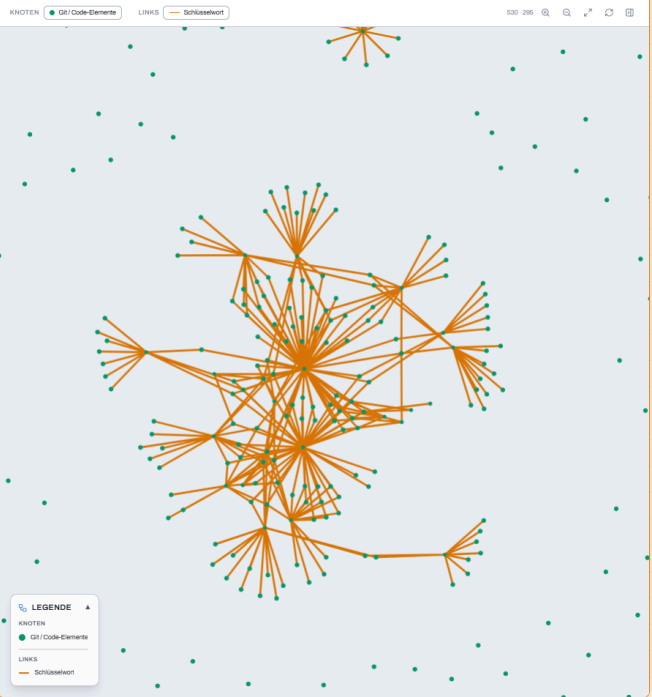
KNOTEN

GR / Code-Elemente

LINKS

Schlüsselwort

530 285



LEGENDE

KNOTEN

GR / Code-Elemente

LINKS

Schlüsselwort

PRODUKTANSICHT 05

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

ANSICHTEN

NEUER CHAT

VERLAUF

Was steht in der README?

Was macht dieses Program?

Was macht FIN-DATA.cpy?

Was ist das für ein Projekt?

WISSENSQUELLEN

Keine Wissensquellen angezeigt

Administrator

CHAT-ANSICHT

Kein Objekt fokussiert

Projekt: COBOLProjekt

Wo wird auf diese DB2-Tabelle zugegriffen?

COBOL-Programm erklären

Aufbau, Sections/Paragraphen, Copybooks und Aufgabe eines COBOL-Moduls

Auflaufkette verfolgen

Wie ruft dieses Programm auf (CALL, JCL-Step, CICS-Transaktion), was ruft es selbst auf?

Projekts Onboarding starten

Briefing, Zusammenfassung, Dokumente, offene Punkte — rollenabhängig

Datenfeld finden

Wie wird ein Feld (Copybook, VSAM-Record, DB2-Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: KONTO-DATEN.cpy.d

Frage Doctus AI...

gpt-5.6-luna

WIKI: KEIN FEHLER BACHEL. BITTE ÜBERPRÜFE SENSIBILE INFORMATIONEN WERD AUF IHRE RICHTIGKEIT.

WISSENSNETZ (GRAPH)

Kein Objekt fokussiert

KNOTEN

GR / Code-Elemente

LINKS

Schlüsselwort

530 285

Suche

Zoom

Reset

Grid

LEGENDE

KNOTEN

GR / Code-Elemente

LINKS

Schlüsselwort

Sitzung entfernt

PRODUKTANSICHT 06

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

+ Ansicht hinzufügen

NEUER CHAT

CHAT-ANSICHT

Kein Objekt fokussiert

Projekt: COBOLProjekt

Wo wird auf diese DB2-Tabelle zugegriffen?

COBOL-Programm erklären
Aufbau, Sections/Paragraphen, Copybooks und
Aufgabe eines COBOL-Moduls

Auflaufkette verfolgen
Wer ruft dieses Programm auf (CALL, JCL-Step,
CICS-Transaktion), was ruft es selbst auf?

Projekts Onboarding starten
Briefing, Zusammenfassung, Dokumente, offene
Punkte — rollenspezifisch

Datenfeld finden
Wo wird ein Feld (Copybook, VSAM-Record, DB2-
Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: KONTO-DATEN.cpy.d

Frag Docatus AI...

gpt-5.6-luna

WIKI: KEIN FEHLER BACHEIN. BITTE ÜBERPRÜFE SENSIBILE INFORMATIONEN WIEDER AUF IHRE RICHTIGKEIT!

WISSENSQUELLEN

Keine Wissensquellen angezeigt

Administrator
Lukas Stang

WISSENSNETZ (GRAPH)

KONTOTABELLE - PROGRAM

KNOTEN GR / Code-Elemente LINKS Schlüsselwort

LEGENDE

KNOTEN GR / Code-Elemente

LINKS Schlüsselwort

Dokument

KONTO-DATEN.cpy

Quelle

Verbindungen

- KONTO-DATEN
- KONTO-STAMM
- KONTO-NR
- KONTO-BEZEICHNUNG
- KONTO-ABRECHNUNG
- KONTO-INHABER-NAME
- KONTO-SALDEN
- KONTO-TYP
- KONTO-GROBKONTO
- KONTO-SPARKONTO

In passender Ansicht Öffnen

Auf diesen Knoten fokussieren

Verknüpfung erstellen

PRODUKTANSICHT 07

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

ANSICHTEN

NEUER CHAT

VERLAUF

- Erkläre mir das Programm START
- Was passiert hier?
- Was steht in der README?
- Was macht dieses Programm?
- Was macht FIN DATA.cpy?
- Was ist das für ein Projekt?

WISSENSQUELLEN

Keine Wissensquellen angezeigt

Administrator

- Leitend Steuerung

CHAT-ANSICHT

Projekt: COBOLProjekt

Was passiert in dieser CICS-Transaktion?

COBOL Programm erklären

- Aufbau, Sections/Paragraphen, Copybooks und Aufgabe eines COBOL-Moduls

Auftragskette verfolgen

- Wie ruft dieses Programm auf (CALL, JCL-Step, CICS-Transaktion), was ruft es selbst auf?

Projekt Onboarding starten

- Drehing: Zusammenfassung, Dokumente, offene Punkte — rollenabhängig

Datenfeld finden

- Wie wird ein Feld (Copybook, VSAM-Record, DB2-Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: SYS-CONFIG

Frage Doctus AI...

+

gpt-5.6-luna

KEINER FEHLER BACHEN. BITTE ÜBERPRÜFE SENSIBLE INFORMATIONEN WIEDER AUF IHRE RICHTIGKEIT!

CODE-EDITOR

copy/SYS-CONFIG.cpy

```
1  * SYSTEM CONFIGURATION
2  01  SYS-CONF-
3      05  SYS-NAME          PIC X(20) VALUE "COBOL-TEST-SYSTEM"
4      05  SYS-VERSION       PIC X(05) VALUE "V2.0".
5      05  SYS-ENVIRONMENT    PIC X(10) VALUE "DEVELOPMENT".
6      05  SYS-ANAL-TIMEZONE  PIC S(02) VALUE 04.
7      05  SYS-LOG-PATH       PIC X(50) VALUE "/var/log/cobol/".
8      05  SYS-TIMEOUT        PIC S(03) VALUE 120.
9      05  SYS-CLASS          PIC X(10).
10     05  SYS-DEBUG-ON       VALUE "DEBUG".
11     05  SYS-AUDIT-ON       VALUE "AUDIT".
12
```

DOKUMENTEN: 9 CODE-REFERENZEN

VERWENDET VON

- MAINPROG
 - zsc/MAINPROG.cbl

VERKNÜPFTE DOKUMENTE

- README.md
 - git
- DEVELOPER_DOC.md
 - git
- SYS-CONFIG.cpy
 - git

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

RECHERCHER CHAT

VERLAUF

Erkläre mir das Programm START

Was passiert hier?

Was steht in der README?

Was macht dieses Programm?

Was macht FIN DATA.cpy?

Was ist das für ein Projekt?

WISSENSQUELLEN

Keine Wissensquellen angezeigt

Administrator

Logout

CHAT-ANSICHT

Projekt: COBOLProjekt

Was passiert in dieser CICS-Transaktion?

COBOL Programm erklären

Aufbau, Sections/Paragraphen, Copybooks und Aufgabe eines COBOL-Moduls

Auftritte verfolgen

Wie ruft dieses Programm auf (CALL, JCL-Step, CICS-Transaktion), was ruft es selbst auf?

Projekte Onboarding starten

Briefing, Zusammenfassung, Dokumente, offene Punkte — rollensabhängig

Datenfeld finden

Wie wird ein Feld (Copybook, VSAM-Record, DB2-Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: SYS-CONFIG

Frag Docatus AI...

+

gpt-5.6-luna

WIKI: KEIN FEHLER BACHEL. BITTE ÜBERPRÜFE SENSIBLE INFORMATIONEN WIEDER AUF IHRE RICHTIGKEIT!

CODE-EDITOR

copy/SYS-CONFIG.cpy

1

2

3

4

5

6

7

8

9

10

11

12

* SYSTEM CONFIGURATION

01 SYS-CONF-01.

05 SYS-NAME PIC X(20) VALUE "COBOL-TEST-SYSTEM".

05 SYS-VERSION PIC X(05) VALUE "V2.0".

05 SYS-ENVIRONMENT PIC X(10) VALUE "DEVELOPMENT".

05 SYS-MAX-TIMEOUTS PIC S(02) VALUE 04.

05 SYS-LOG-PATH PIC X(50) VALUE "/var/log/cobol/".

05 SYS-TIMEOUT PIC S(03) VALUE 120.

05 SYS-CLASS PIC X(10).

05 SYS-DEBUG-ON VALUE "DEBUG".

05 SYS-AUDIT-ON VALUE "AUDIT".

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

RECHERCHER

VERLAUF

Erkläre mir das Programm START

Was passiert hier?

Was steht in der README?

Was macht dieses Programm?

Was macht FIN DATA.cpy?

Was ist das für ein Projekt?

WISSENSQUELLEN

Keine Wissensquellen angezeigt

Administrator

Logout

CHAT-ANSICHT

Projekt: COBOLProjekt

Was passiert in dieser CICS-Transaktion?

COBOL Programm erklären

Aufbau, Sections/Paragraphen, Copybooks und Aufgabe eines COBOL-Moduls

Auftragskette verfolgen

Wie ruft dieses Programm auf (CALL, JCL-Step, CICS-Transaktion), was ruft es selbst auf?

Projekte Onboarding starten

Briefing, Zusammenfassung, Dokumente, offene Punkte — rollenabhängig

Datenfeld finden

Wie wird ein Feld (Copybook, VSAM-Record, DB2-Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: SYS-CONFIG

Frage Doctus AI...

gpt-5.6-luna

RYAN: FEHLER BACHEL. BITTE ÜBERPRÜFE SENSIBLE INFORMATIONEN WIEDER AUF IHRE RICHTIGKEIT!

CODE-EDITOR

copy/SYS-CONFIG.cpy

1

2

3

4

5

6

7

8

9

10

11

12

SYSTEM CONFIGURATION

01 SYS-CONF-20.

02

03

04

05

06

07

08

09

10

11

12

PIC X(20) VALUE "COBOL-TEST-SYSTEM".

PIC X(05) VALUE "V2.0".

PIC X(10) VALUE "DEVELOPMENT".

PIC S(02) VALUE 04.

PIC X(50) VALUE "/var/log/cobol/".

PIC S(03) VALUE 120.

PIC X(10).

VALUE "DEBUG".

VALUE "AUDIT".

COBOLPROJEKT

Programme, Paragraph oder Dokument suchen... (Strg+K)

CHAT-ANSICHT

Projekt: COBOLProjekt

COBOLPROJEKT

SYS-CONFIG1.CPY

ME

was wird in Zeile 3 gemacht?

In Zeile 3 wird das COBOL-Datenfeld **SYS-NAME** definiert:

- 95** : Hierarchieebene innerhalb einer übergeordneten Struktur
- PIC X(20)** : alphanumerisches Feld mit 20 Zeichen
- VALUE "COBOL-TEST-SYSTEM"** : Initialwert des Feldes

Das Feld enthält somit den Namen des Systems „COBOL-TEST-SYSTEM“.

REFERenzierte Quellen:

SYS-CONFIG1.CPY 13-3

Fokus: COBOLProjekt

Pin: SYS-NAME

Frag Docus AI...

+

gpt-5.5-luna

Kleiner Fehler! Bitte überprüfe diese Informationen immer auf ihre Richtigkeit!

CODE-EDITOR

copy/SYS-CONFIG1.cpy

REFERENZEN

C:22

AREA 8 (10-75)

1

2

3

4

5

6

7

8

9

10

11

12

* SYSTEM CONFIGURATION

01 SYS-CONFIG.

05 SYS-NAME PIC X(20) VALUE "COBOL-TEST-SYSTEM".

05 SYS-VERSION PIC X(85) VALUE "V2.0".

05 SYS-ENVIRONMENT PIC X(10) VALUE "DEVELOPMENT".

05 SYS-MAX-TIMEOUTS PIC 9(82) VALUE 04.

05 SYS-LOG-PATH PIC X(50) VALUE "/var/log/cobol/".

05 SYS-TIMEOUT PIC 9(83) VALUE 120.

05 SYS-FLAGS PIC X(10).

08 SYS-DEBUG-ON VALUE "DEBUG".

88 SYS-AUDIT-ON VALUE "AUDIT".

KONFIGURATION

Projekte

Wissensquellen

Teams

Nutzer

AI-Paramotor

Logs & Status

Layout & Design

PROJEKTE

AKTIVE PROJEKTE

Neues Projekt

COBOLProjekt

aktiv

Aktiv

VERKNÜPfte WISSENSQUELLEN

COBOLTestRepository (Git)

Diabus_Anforderungskonzept.pdf (Local)

IP-Covered: 82,188,238,189

Schließen

KONFIGURATION

Projekte

Wissensquellen

Teams

Nutzer

AI-Parameter

Logs & Status

Layout & Design

WISSENSQUELLEN

WISSENSQUELLEN & INTEGRATIONEN

2 Quellen

Erweitere die RAG-Wissensbasis von Doctus um zusätzliche Firmenquellen neben Git-Code.

FILTER: COBOLProjekt

QUELLEN-NETZWERK

DOCTUS

COBOLProjekt

CobolTestRepository

Doctus_Anforderung...

VERBUNDENE DATENQUELLEN (2)

CobolTestRepository

Git

Erfolgreich

Projekt: COBOLProjekt

Branch: main

Aktiv (letzter Sync: 11.08., 17:34)

KONTEXT-NOTIZ FÜR DIE KI

z. B. "Diese Seiten heißen bei uns Schlüsselbeschreibungen und bedeuten ..."

INTERVALL: Nur manuell

Doctus_Anforderungskonzept.pdf

LOCAL

Fehler

Projekt: COBOLProjekt

Fehler beim Sync

KONTEXT-NOTIZ FÜR DIE KI

z. B. "Diese Seiten heißen bei uns Schlüsselbeschreibungen und bedeuten ..."

MANUELLES DOKUMENT

NEUE WISSENSQUELLE ANBINDEN

Git

Code-Repositories (GitHub, GitLab, Bitbucket) klonen und durchsuchbar machen.

Confluence

Spaces und Knowledge Base Artikel indizieren.

Jira Software

Tickets, Epics und Fehlerberichte verknüpfen.

IP-Connid: 82.156.238.150

Schließen

PRODUKTANSICHT 14

KONFIGURATION

Projekte

Wissensquellen

Teams

Nutzer

AI-Parametrier

Logs & Status

Layout & Design

TEAMS

NEUES TEAM

z.B. Tragwerksplanung

+ Team anlegen

> Default Team

Schließen

PRODUKTANSICHT 15

KONFIGURATION

Projekte

Wissensquellen

Teams

Nutzer

AI-Parameter

Logs & Status

Layout & Design

NUTZER

BENUTZERNAME

z.B. m.mustermann

NAME (OPTIONAL)

z.B. Max Mustermann

ROLLE

Nutzer

+ Anlegen

Das Startpasswort wird automatisch erzeugt, hier einmalig angezeigt und muss beim ersten Login geändert werden.

admin Administrator zuletzt angemeldet 21.8.2026, 09:17:10	Administrator
e2e-tester — zuletzt angemeldet 31.7.2026, 23:56:22	Administrator
m.mustermann Max Mustermann zuletzt angemeldet 31.7.2026, 15:54:03	Nutzer
Testnutzer deaktiviert — noch nie angemeldet	Nutzer
testuser2 Test User 2 zuletzt angemeldet 10.8.2026, 22:19:10	Nutzer

Schließen

KONFIGURATION

Projekte

Wissensquellen

Teams

Nutzer

AI-Parameter

Logs & Status

Layout & Design

AI-PARAMETER

KI-MODELLPROFIL & API-EINSTELLUNGEN

Hier kannst du mehrere LLM-Modelle parallel konfigurieren. Im Chat-Fenster kannst du dann einfach per Dropdown zwischen ihnen umschalten.

KONFIGURIERTE MODELLPROFILE

Profil hinzufügen

Lokales Ollama (Default)

OLLAMA • ollama:3.1.1:30

dots-3-note-preview

OPENAI • dots-studio/dots-3-note-provider:free • https://openrouter...

gpt-5.6-luna

AKTIV

OPENAI • gpt-5.6-luna

AKTIVES PROFIL / GEESETZES PROFIL (MIT 0 COMPLIANCES)

gpt-5.6-luna (gpt-5.6-luna)

KINGARTISTES (TEMP)

0.7

SYSTEM PROMPT

Du bist Doctus, ein Enterprise-Wissensassistent. Du hilfst dabei, große, gewachsene Projektfeldschaften zu verstehen -- von COBOL-Beständen über Copybooks und JCL bis zu Dokumenten und angebundenen Wissensquellen (z. B. Confluence, Jira). Antworte präzise und begründet, stütze dich ausschließlich auf die dir bereitgestellten und indextierten Inhalte, und mache transparent, wenn dir Informationen fehlen oder unsicher sind.

AI-Einstellungen speichern

IPControl: 82.158.238.150

Schließen

PRODUKTANSICHT 17

KONFIGURATION

Projekte

Wissensquellen

Teams

Nutzer

AI-Parameter

Logs & Status

Layout & Design

LOGS & STATUS

SYSTEMUMGEBUNG & STATUS

FASTAPI BACKEND-API

http://82.165.216.180:8080

ONLINE

OLLAMA LLM SERVICE

http://ollama:11434

ONLINE

POSTGRES VECTOR DB

postgres://admin:***@db:5432/doctus

SECURE

REDIS CELERY BROKER

redis://redis:6379/0

VERBUNDEN

DIAGNOSE-BUNDLE

Sammlt relevante DB-Metadaten und Service Logs in einem heruntergeladenen Archiv für den Support — nur für Administratoren.

Bundle erzeugen

INDIZIERUNGS-LOGS DER WISSENSQUELLEN

CobolTestRepository

SPR

Letzter Sync: 11.8.2026, 17:31:11

Erfolgreich

Log

Doctus_Anforderungskonzept.pdf

LOCAL

Letzter Sync: Nie

Fehler

Log

IP-Connid: 82.165.216.180

Schließen

PRODUKTANSICHT 18

KONFIGURATION

Projekte

Wissensquellen

Teams

Nutzer

AI-Parameter

Logs & Status

Layout & Design

LAYOUT & DESIGN

FARBTHEMA

Helimodus aktivieren

Wechselt zwischen dunkler und heller Benutzeroberfläche.

SPRACHE

Sprache

Wähle die Anzeigesprache der Benutzeroberfläche.

Deutsch

Englisch

WORKSPACE LAYOUT SPLIT

Schmäler Chat

40% Chat / 60% Code

Standard

40% Chat / 60% Code

Gleichmäßig

50% Chat / 50% Code

Breiter Chat

80% Chat / 20% Code

MONACO EDITOR OPTIONEN

SCHRIFTGRÖßE (PX)

13px

SCHRIFTART

JetBrains Mono

Minimap anzeigen

Kompakte Scroll-Übersicht für längeren Code.

WISSENSGRAPH EXPORT

Neo4j Cypher-Export

Exportiere den gesamten Wissensgraphen als Neo4j Cypher-Skript.

Neo4j Export

IP-ConnId: 82.188.238.180

Schließen

PRODUKTANSICHT 19

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

CHAT-ANSICHT

SYS-NAME

DATA-ITEM

Projekt: COBOLProjekt

Welche Programme sind noch nicht dokumentiert?

COBOL-Programmerklären

Aufbau, Sections/Paragraphen, Copybooks und
Aufgabe eines COBOL-Moduls

Aufaufrufe verfolgen

Wer ruft dieses Programm auf (CALL, JCL-Step,
CICS-Transaktion), was ruft es selbst auf?

Projekts Onboarding starten

Struktur, Zusammenfassung, Dokumente, offene
Punkte — rollenspezifisch

Datenfeld finden

Wo wird ein Feld (Copybook, VSAM-Record, DB2-
Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: SYS-NAME

Frag Docatus AI...

+

gpt-5.6-luna

↩

KOMM FEHLER BACHEN. BITTE ÜBERPRÜFE SENSIBLE INFORMATIONEN IMMER AUF IHRE RICHTIGKEIT.

WISSENSNETZ (GRAPH)

Kein Objekt fokussiert

KNOTEN

GR/Code-Elemente

LINKS

Schlüsselwort

ANSICHT AUSWÄHLEN

Code-Editor

Dokumentation

Wissensnetz (Graph)

Call-Graph

Web-Vorschau

Link Manager

LEGENDE

KNOTEN

GR/Code-Elemente

LINKS

Schlüsselwort

PRODUKTANSICHT 20

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

CHAT-ANSICHT

Projekt: COBOLProjekt

Welche Programme sind noch nicht dokumentiert?

**COBOL Programm erklären**
Aufbau, Sections/Paragraphen, Copybooks und
Aufgabe eines COBOL-Moduls

**Aufaufliste verfolgen**
Wer ruft dieses Programm auf (CALL, JCL-Step,
CICS-Transaktion), was ruft es selbst auf?

**Projekt Onboarding starten**
Briefing, Zusammenfassung, Dokumente, offene
Punkte – rollensabhängig

**Datenfeld finden**
Wo wird ein Feld (Copybook, VSAM-Record, DED-
Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: SYS-NAME

Frage Docatus AI...

gpi-5.6-luna

RIKARD FEHLER BÄCKEN. BITTE ÜBERPRÜFE SENSIBLE INFORMATIONEN IMMER AUF IHRE RICHTIGKEIT!

LINK MANAGER

COBOLProjekt

gpi-5.6-luna (gp...

48 Links

Topics

% 35 %

Automatisch verknüpfen

Manuell

Suchen...

Alle %

Alle 100 % bestätigen

Offen

set

Bestätigt

298

Abgelehnt

9

Alle

Code == Doc

Doc == Doc

WS-HAUPTFEHLER-KODE

data_item = srcKONTOABRECHNUNG.cbl:55

100%

keyword

Beschreibung

WS-KONTEN-FEHLER

data_item = srcKONTOABRECHNUNG.cbl:53

100%

keyword

Beschreibung

WS-KONTEN-VERARBEITET

data_item = srcKONTOABRECHNUNG.cbl:52

100%

README.md

keyword

Beschreibung

WS-ABRECHNUNGS-MONAT

data_item = srcKONTOABRECHNUNG.cbl:48

100%

ABRECHNUNGSLAUF.jcl

keyword

Beschreibung

WS-ABRECHNUNGS-LAUF-ID

data_item = srcKONTOABRECHNUNG.cbl:48

100%

README.md

keyword

Beschreibung

WS-ABRECHNUNGS-LAUF-ID

data_item = srcKONTOABRECHNUNG.cbl:48

100%

ABRECHNUNGS-LAUF.jcl

keyword

Beschreibung

DATEI-ENDE

data_item = srcKONTOABRECHNUNG.cbl:45

100%

KONTOABRECHNUNG.cbl

keyword

Beschreibung

WS-KONTO-DATEI-STATUS

data_item = srcKONTOABRECHNUNG.cbl:42

100%

KONTOABRECHNUNG.cbl

keyword

Beschreibung

AUSZUG-DATENSATZ

data_item = srcKONTOABRECHNUNG.cbl:34

100%

README.md

keyword

Beschreibung

AUSZUG-DATEI

file_id = srcKONTOABRECHNUNG.cbl:33

100%

KONTOABRECHNUNG.cbl

keyword

Beschreibung

AUSZUG-DATEI

file_id = srcKONTOABRECHNUNG.cbl:33

100%

README.md

keyword

Beschreibung

KONTO-DATEI

file_id = srcKONTOABRECHNUNG.cbl:30

100%

KONTOABRECHNUNG.cbl

keyword

Beschreibung

PRODUKTANSICHT 21

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

Ansicht hinzufügen

CHAT-ANSICHT

Projekt: COBOLProjekt

Wie hängen diese beiden Module zusammen?

COBOL-Programm erklären
Aufbau, Sections/Paragraphen, Copybooks und
Aufgabe eines COBOL-Moduls

Auflufkette verfolgen
Wann ruft dieses Programm auf (CALL, JCL-Step,
CICS-Transaktion), was ruft es selbst auf?

Projekt Onboarding starten
Briefing, Zusammenfassung, Dokumente, offene
Punkte — rollenabhängig

Datenfeld finden
Wo wird ein Feld (Copybook, VSAM-Record, DB2-
Tabelle) definiert und wo verwendet?

Fokus: COBOLProjekt

Pin: VERFAHREN.cbl:75

Frage Doctus AI...

+ gpt-5.0-luna

Kann Fehler machen. Bitte überprüfe sensible Informationen immer auf ihre Richtigkeit.

CALL-GRAPH

VERFAHREN
CALL-GRAPH: 2 HOPS

CALL

PERFORM

GOTO

COPY

CONTAINS

JSON

CSV

GRAPHML

CODE-EDITOR

VERFAHREN - PROGRAM

src/VERFAHREN.cbl

SEQUENZ (1-4)

```
48 300-STEP-INITIALIZE.  
49  ADD 1 TO WS-STEP-COUNT.  
50  MOVE "INITIALIZATION" TO WS-CURRENT-STEP.  
51  DISPLAY "STEP " WS-STEP-COUNT ": " WS-CURRENT-STEP.  
52  
53  MOVE "INFO " TO WS-LOG-LEVEL.  
54  MOVE "EXECUTING INITIALIZATION" TO WS-LOG-MSG.  
55  CALL "LOGGER" USING WS-LOG-DATA.  
56  
57  MOVE "SYSTEM STARTUP" TO WS-MSG.  
58  CALL "UTILPROG" USING WS-MSG WS-RC.  
59  
60 350-STEP-DB-LOAD.  
61  ADD 1 TO WS-STEP-COUNT.  
62  MOVE "DB LOAD" TO WS-CURRENT-STEP.  
63  DISPLAY "STEP " WS-STEP-COUNT ": " WS-CURRENT-STEP.  
64  
65  MOVE "GET" TO WS-DB-ACTION.  
66  MOVE "BATCH-JOB-DATA" TO DB-KEY OF WS-DB-REC.  
67  CALL "DBPROG" USING WS-DB-ACTION WS-DB-REC.  
68  
69  IF DB-OK OF WS-DB-REC  
70  | DISPLAY "DB LOADED: " DB-DATA-FIELD-1 OF WS-DB-REC  
71  ELSE  
72  | DISPLAY "DB LOAD FAILED"  
73  END-IF.  
74  
75 200-STEP-MAIN-LOGIC.  
76  ADD 1 TO WS-STEP-COUNT.  
77  MOVE "MAIN LOGIC" TO WS-CURRENT-STEP.  
78  DISPLAY "STEP " WS-STEP-COUNT ": " WS-CURRENT-STEP.  
79  
80  MOVE "INFO " TO WS-LOG-LEVEL.  
81  MOVE "EXECUTING MAIN LOGIC" TO WS-LOG-MSG.  
82  CALL "LOGGER" USING WS-LOG-DATA.  
83  
84  CALL "MAINPROG".  
85  
86 350-STEP-DB-UPDATE.  
87  ADD 1 TO WS-STEP-COUNT.  
88  MOVE "DB UPDATE" TO WS-CURRENT-STEP.  
89  DISPLAY "STEP " WS-STEP-COUNT ": " WS-CURRENT-STEP.  
90  
91  MOVE "PUT" TO WS-DB-ACTION.  
92  MOVE "BATCH-JOB-RESULT" TO DB-KEY OF WS-DB-REC.  
93  MOVE "SUCCESSFUL RUN" TO DB-DATA-FIELD-1 OF WS-DB-REC.  
94  CALL "DBPROG" USING WS-DB-ACTION WS-DB-REC.  
95  
96 300-STEP-FILE-REPORT.  
97  ADD 1 TO WS-STEP-COUNT.  
98  MOVE "FILE REPORTING" TO WS-CURRENT-STEP.  
99  DISPLAY "STEP " WS-STEP-COUNT ": " WS-CURRENT-STEP.  
100  
101  MOVE "INFO " TO WS-LOG-LEVEL.
```

PRODUKTANSICHT 22

LEGACY INTELLIGENCE WORKSPACE

Verstehe die Systeme, die alles am Laufen halten.

NACHVERFOLGEN

ERKLÄREN

MODERNISIEREN

Anmelden

Bitte mit Benutzernamen und Passwort anmelden.

Benutzername

Passwort

Anmelden →

COBOLPROJEKT

Program, Paragraph oder Dokument suchen... (Strg+K)

+ Ansicht hinzufügen

CODE-EDITOR

PROGRAM

VERFAHREN - PROGRAM

VERFAHREN - CH1

59 REFERENZEN

SEQUENZ (1-40)

19

PROCEDURE DIVISION.

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

MOVE "INFO " TO WS-LOG-LEVEL.

MOVE "VERFAHREN" TO WS-LOG-SENDER.

MOVE "STARTING COMPLEX BATCH SEQUENCE" TO WS-LOG-HSG.

CALL "LOGGER" USING WS-LOG-DATA.

DISPLAY "=====

DISPLAY "VERFAHREN: STARTING COMPLEX BATCH SEQUENCE".

DISPLAY "=====

PERFORM 100-STEP-INITIALIZE.

PERFORM 150-STEP-DB-LOAD.

PERFORM 200-STEP-MAIN-LOGIC.

PERFORM 250-STEP-DB-UPDATE.

PERFORM 300-STEP-FILE-REPORT.

MOVE "INFO " TO WS-LOG-LEVEL.

MOVE "BATCH SEQUENCE COMPLETED" TO WS-LOG-HSG.

CALL "LOGGER" USING WS-LOG-DATA.

DISPLAY "=====

DISPLAY "VERFAHREN: SEQUENCE COMPLETED".

DISPLAY "=====

STOP RUN.

STEPS SECTION.

100-STEP-INITIALIZE.

ADD 1 TO WS-STEP-COUNT.

MOVE "INITIALIZATION" TO WS-CURRENT-STEP.

DISPLAY "STEP " WS-STEP-COUNT ": " WS-CURRENT-STEP.

MOVE "INFO " TO WS-LOG-LEVEL.

MOVE "EXECUTING INITIALIZATION" TO WS-LOG-HSG.

CALL "LOGGER" USING WS-LOG-DATA.

MOVE "SYSTEM STARTUP" TO WS-HSG.

CALL "UTILPROG" USING WS-HSG WS-RC.

150-STEP-DB-LOAD.

ADD 1 TO WS-STEP-COUNT.

MOVE "DB LOAD" TO WS-CURRENT-STEP.

DISPLAY "STEP " WS-STEP-COUNT ": " WS-CURRENT-STEP.

MOVE "GET" TO WS-DB-ACTION.

MOVE "BATCH-JOB-DATA" TO DB-KEY OF WS-DB-REC.

CALL "DBPROG" USING WS-DB-ACTION WS-DB-REC.

IF DB-OK OF WS-DB-REC

DISPLAY "DB LOADED: " DB-DATA-FIELD-1 OF WS-DB-REC

ELSE

DISPLAY "DB LOAD FAILED"

DOKUMENTATION

DEVELOPER_DOC.md

60 REFERENZEN

DEVELOPER_DOC.md

Wissensquelle • Dokumenten-Viewer

Entwicklerdokumentation: CobolTestRepository

Diese Dokumentation bietet einen umfassenden Überblick über das **CobolTestRepository**. Sie beschreibt jedes Programm, jede Sektion, jeden Paragraphen und jedes Datenfeld im Detail. Das Repository ist ein modular aufgebautes COBOL-System, das Batch-Verarbeitung, Datenbank-Simulationen, Datei-I/O und Geschäftsflogik kombiniert.

1. Gemeinsame Datenstrukturen (Copybooks)

Die folgenden Copybooks definieren die globalen Datenstrukturen, die von mehreren Programmen über **COPY** -Anweisungen eingebunden werden.

CONSTANTS.cpy - Anwendungsübergreifende Konstanten

Dieses Copybook enthält zentrale Informationen zur Anwendung und Rückgabecodes.

Feldname	Level	PIC	Beschreibung
WS-APP-INFO	01	-	Gruppierung der Anwendungsinformationen.
APP-NAME	05	X(20)	Name der Anwendung ("COBOL-DEMO-APP").
VERSION	05	X(05)	Aktuelle Versionsnummer ("1.0.1").
BUILD-DATE	05	X(10)	Datum des letzten Builds ("2025-06-13").
WS-RETURN-CODES	01	-	Standardisierte Rückgabewerte.
SUCCESS-CODE	05	9(01)	Erfolgreiche Ausführung (0).
ERROR-CODE	05	9(01)	Allgemeiner Fehler (1).
FATAL-CODE	05	9(01)	Kritischer Systemfehler (9).
WS-FILE-STATUS-CONST	01	-	Konstanten für Dateioperationen.
FS-OK	05	X(02)	Dateioperation erfolgreich ("00").
FS-EOF	05	X(02)	Ende der Datei erreicht ("10").
FS-NOT-FOUND	05	X(02)	Datei nicht gefunden ("23").

USER_DATA.cpy - Benutzerdatenstruktur

Definiert die Struktur eines Benutzerdatensatzes.